

Mon (25th Nov)

4 problems

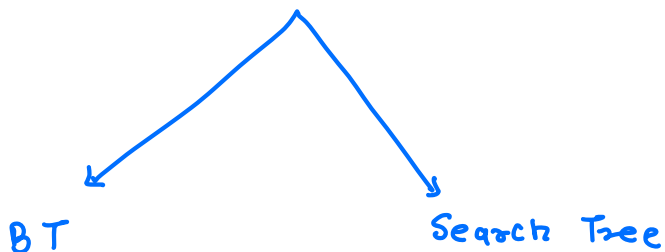
Contest

- Searching
- LL
- Stacks
- Queues
- Trees

- Agenda

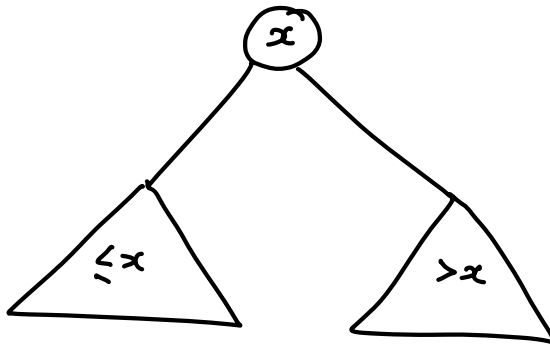
- ✓ BST
- ✓ Searching
- ✓ Insertion
- ✓ Find smallest
- ✓ Find largest
- ✓ Deletion
- ✓ Construct BST
- ✓ Check if BT is BST

Binary Search Tree

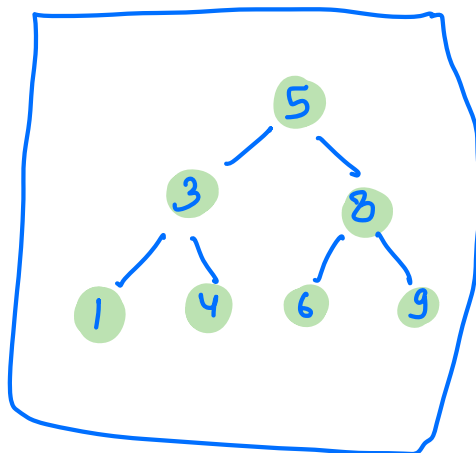


- Searching Efficient
- Data is present in some sorted manner.

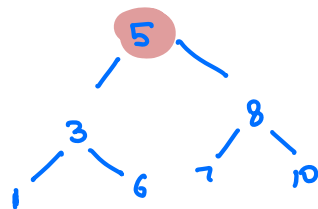
For all nodes:



data in left subtree $< x <$ data in right subtree

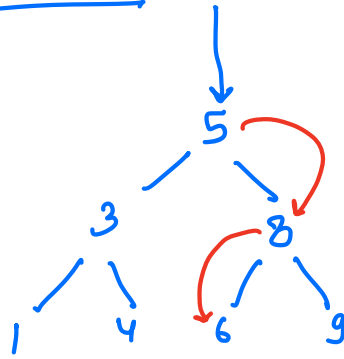


BST ✓



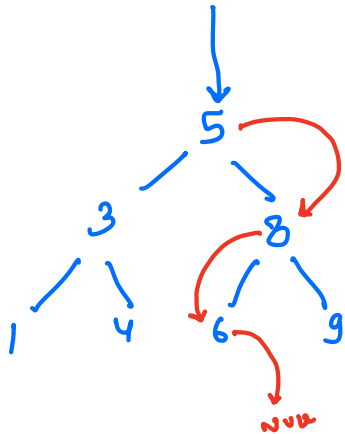
BST ✗

Searching



Search 6

ans = true

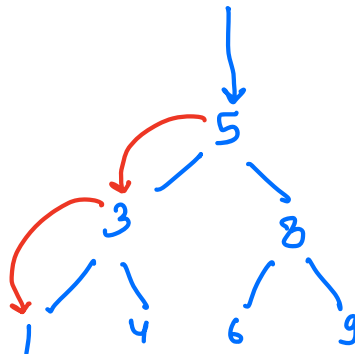


7

ans = False

Quiz 2

No
of
Nodes
visited = 3



1

ans = true

T/F if K is present in the tree or not

boolean Search (Node root, int k)

if (root == NULL) return False

if (root.data == k) return true

if (root.data < k)

return Search (root.right, k)

if (root.data > k)

return Search (root.left, k)

Tc: $O(H)$

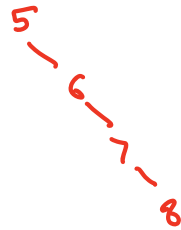
↓

$O(N)$

Sc: $O(H)$

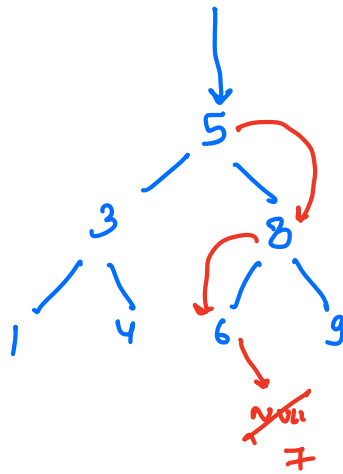
↓

$O(N)$



BST ✓

Insertion in BST



$K = 7$

→ Insert K in given BST
return new root of that tree.

Node Insert($root, K$)

$if (root == null)$

{
Node tmp = new Node(K)
return tmp

$if (root.data == K)$ return root

$if (root.data < K)$

root.right = Insert($root.right, K$)

else

root.left = Insert($root.left, K$)

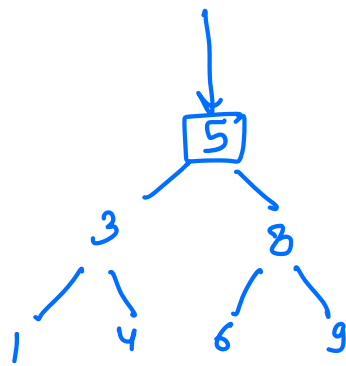
return root

Tc: $O(H)$

↓
 $O(N)$

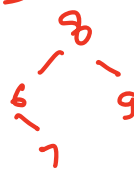
Sc: $O(H)$

↓
 $O(N)$



5 →

add(7)



Insert(8, 7)

root.left = Insert(left, 7)

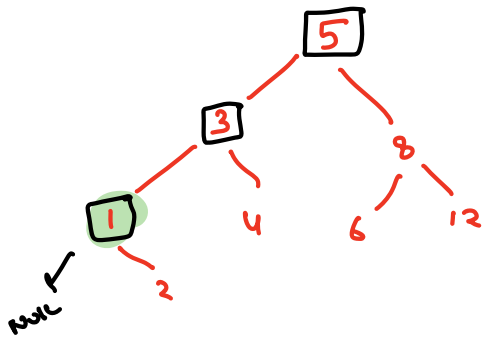
Insert(6, 7)

Insert(right, 7)

Insert(Null, 7)

return tmp





a. Given a BST, Find Smallest element

```

tmp = root
while (tmp.left != null)
{
    tmp = tmp.left
}

return tmp.data
  
```

Tc: $O(H)$
 $\downarrow$
 $O(N)$
 Sc: $O(1)$

a. Given a BST, Find Largest element

```

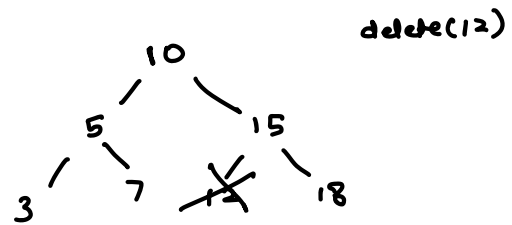
tmp = root
while (tmp.right != null)
{
    tmp = tmp.right
}

return tmp.data
  
```

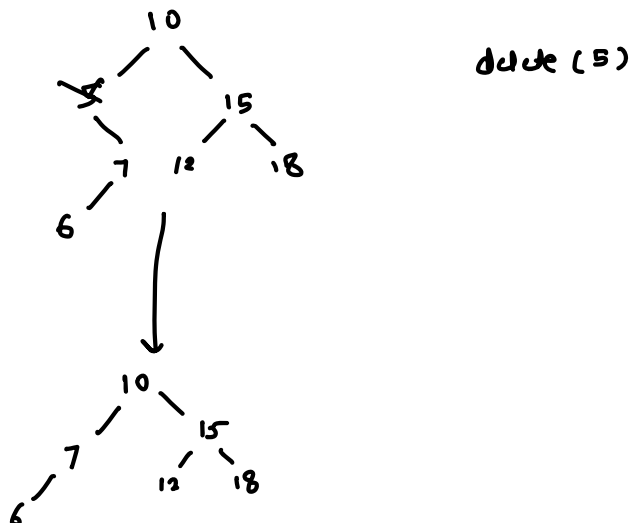
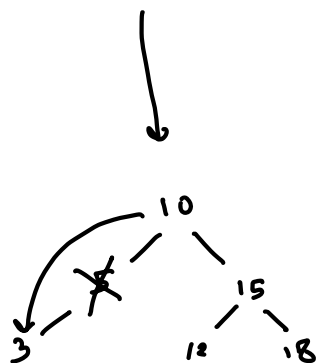
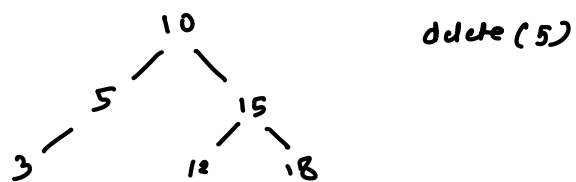
Tc: $O(H)$
 $\downarrow$
 $O(N)$
 Sc: $O(1)$

Deletion in BST

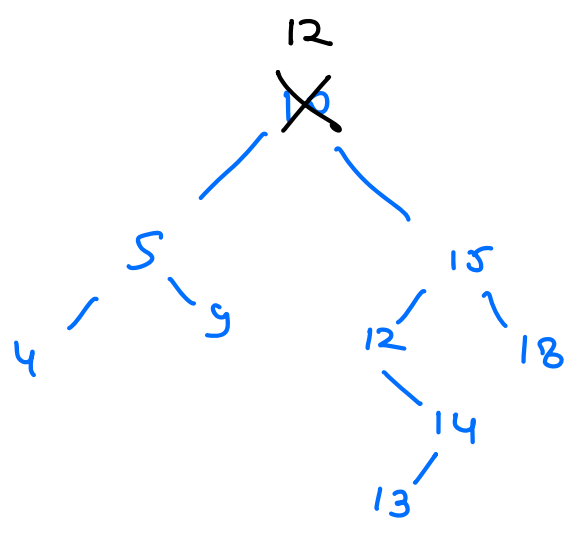
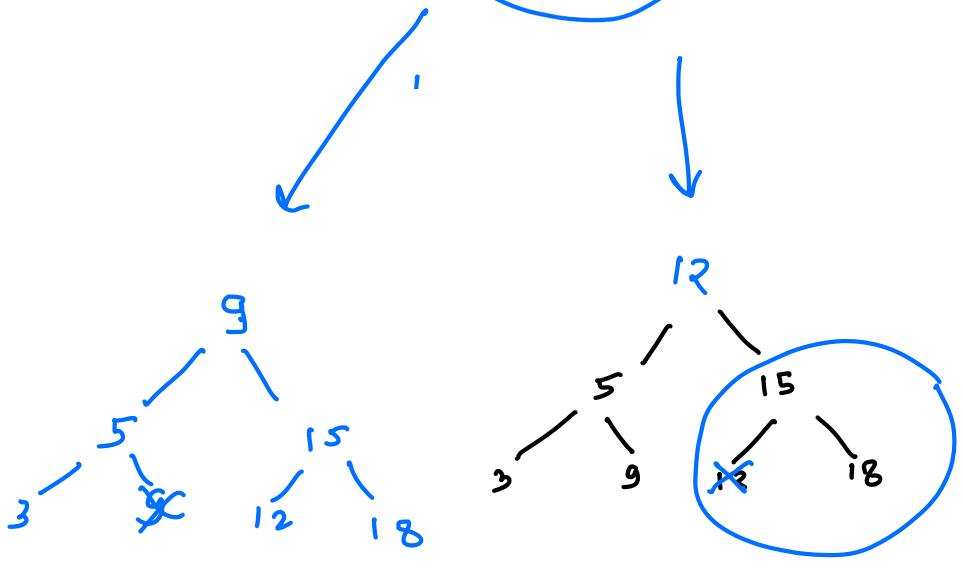
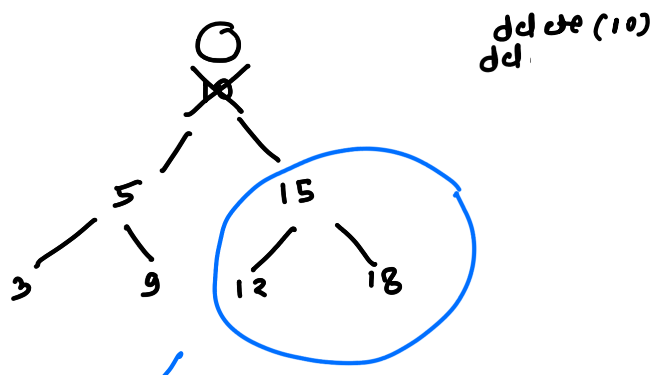
1. Node has zero child



2. Node has 1 child



2. Node has 2 children



// delete K from tree & return new root

Node delete (Root, K)

if (root == null) return null

if (root.data < K)

{ root.right = delete (root.right, K)

else if (root.data > K)

{ root.left = delete (root.left, K)

else

if (root.left == null && root.right == null)

C1 → return null

else if (root.left == null || root.right == null)

C2 → if (root.left == null) return root.right
if (root.right == null) return root.left

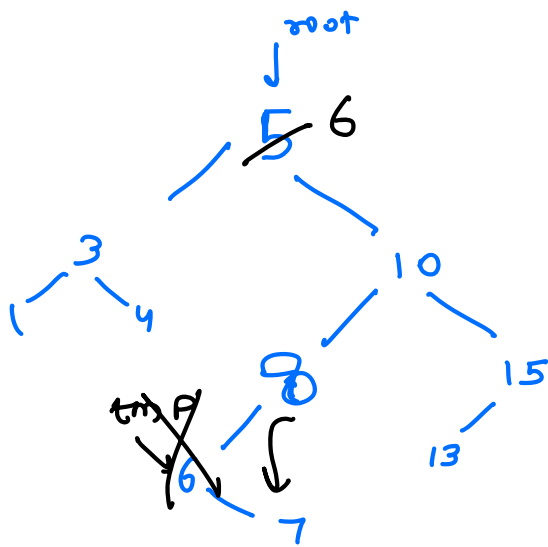
else

C3 → tmp = root.right
while (tmp.left != null)
{ tmp = tmp.left
root.data = tmp.data
root.right = delete (root.right, tmp.data)
return root

smallest in right

T.C: O(H)

↓
O(N)

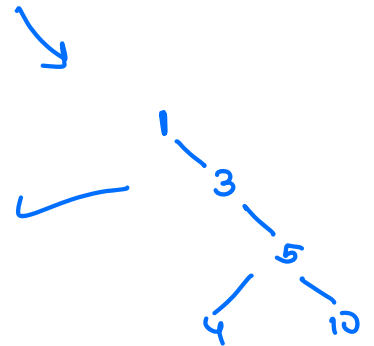


K = 5

Sc: $O(N)$
 $\downarrow$
 $O(N)$

Q. Given N nodes. Construct a BST from it

1, 3, 5, 10, 4



Node root = null

for (i = 0; i < N; i++)

{ root = insert(root, A[i])

return root

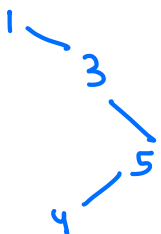
ask(2)

T: $O(N^2)$

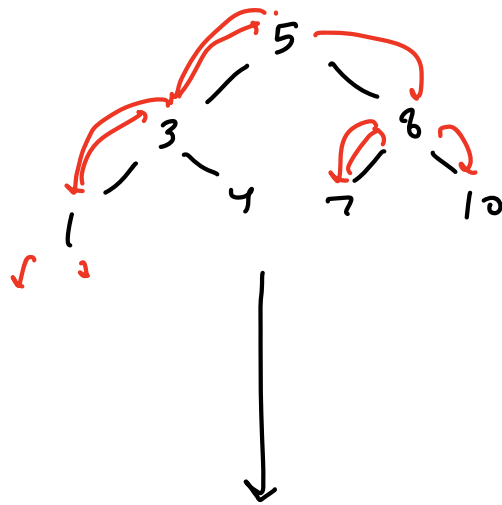
Sc: $O(N)$

without Tree

Sc: $O(N)$



Q. Given a BT. check if it is BST or not



1, 3, 4, 5, 7, 8, 10

if array is sorted
or
not

Inorder of BST is always sorted

TC: $O(N)$

SC: $O(N + H)$

inorder
traversal

↓
recursion
stack

```
class Solver {
```

```
    int prev
```

```
    boolean ans
```

```
    void inorder (root)
```

```
        if (root == null) return
```

```
        inorder (root.left)
```

```
        if (root.data < prev) ans = false
```

```
        prev = root.data
```

```
        inorder (root.right)
```

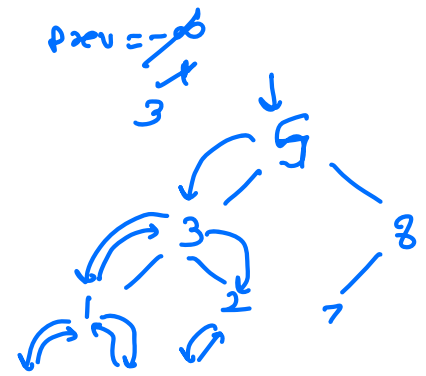
```
main (root)
```

```
    ans = true
```

```
    prev = -∞
```

```
    inorder (root)
```

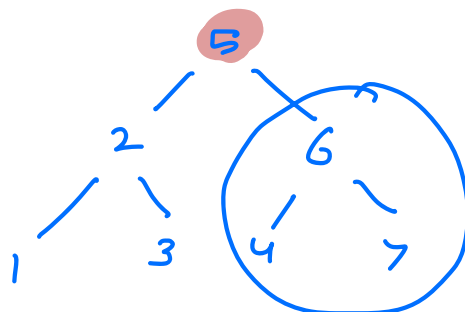
```
    return ans
```



TC: $O(N)$

SC: $O(H)$

↓
 $O(N)$



$5 < 4 \times$